```
# 8-Puzzle using Depth First Search (DFS)

GOAL = "123456780"

# Possible positions for the blank tile
MOVES = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}


# Generate possible next states
def get_neighbors(state):
    blank = state.index("0")

    for move in MOVES[blank]:
        new_state = list(state)

        # Swap blank with the adjacent tile
        new_state[blank], new_state[move] = \
            new_state[move], new_state[blank]

        yield "".join(new_state)


# DFS function
def dfs(start):
    stack = [(start, [start])]
    visited = set()

    while stack:

        state, path = stack.pop()

        if state in visited:
            continue

        visited.add(state)
```

```
        # Goal check
        if state == GOAL:
            return path

        # Generate next states
        for next_state in get_neighbors(state):
            if next_state not in visited:
                stack.append((next_state, path + [next_state]))

    return None


# Display puzzle
def display(path):

    if path is None:
        print("No solution found.")
        return

    print("Number of moves:", len(path) - 1)

    for state in path:
        print()

        for i in range(0, 9, 3):
            row = state[i:i+3]

            print(" ".join(
                "_" if x == "0" else x
                for x in row
            ))


# Main program
start = input("Enter initial state (0 for blank): ")

if len(start) != 9 or not start.isdigit():
    print("Invalid input!")

elif set(start) != set("012345678"):
    print("Enter digits 0 to 8 exactly once.")

else:
    print("\nDFS Solution:")

    solution = dfs(start)
```

display(solution)

```
================== RESTART: C:/Users/Student/Desktop/Om/DFS.py ==================
Enter initial state (0 for blank): 123405678

DFS Solution:
Number of moves: 14232

1 2 3
4 _ 5
6 7 8

1 2 3
4 7 5
6 _ 8

1 2 3
4 7 5
6 8 _

1 2 3
4 7 _
6 8 5

1 2 3
4 _ 7
6 8 5

1 2 3
4 8 7
6 _ 5

1 2 3
4 8 7
6 5 _
```

```
# 8-Puzzle using Iterative Deepening Search (IDS)

GOAL = "123456780"

# Possible positions for the blank tile
MOVES = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}


# Generate possible next states
def get_neighbors(state):
    blank = state.index("0")

    for move in MOVES[blank]:
        new_state = list(state)

        # Swap blank with adjacent tile
        new_state[blank], new_state[move] = \
            new_state[move], new_state[blank]

        yield "".join(new_state)


# Depth Limited Search
def depth_limited_search(state, limit, path, visited):

    # Goal test
    if state == GOAL:
        return path

    # Depth limit reached
    if limit == 0:
        return None

    visited.add(state)

    # Generate next states
```

```
    for next_state in get_neighbors(state):

        if next_state not in visited:

            result = depth_limited_search(
                next_state,
                limit - 1,
                path + [next_state],
                visited
            )

            if result is not None:
                return result

    # Remove state while backtracking
    visited.remove(state)

    return None


# Iterative Deepening Search
def ids(start):

    depth = 0

    while True:

        visited = set()

        solution = depth_limited_search(
            start,
            depth,
            [start],
            visited
        )

        if solution is not None:
            return solution

        depth += 1


# Display puzzle
def display(path):

    if path is None:
```

```
        print("No solution found.")
        return

    print("Number of moves:", len(path) - 1)

    for state in path:
        print()

        for i in range(0, 9, 3):
            row = state[i:i+3]

            print(" ".join(
                "_" if x == "0" else x
                for x in row
            ))


# Main program
start = input("Enter initial state (0 for blank): ")

if len(start) != 9 or not start.isdigit():
    print("Invalid input!")

elif set(start) != set("012345678"):
    print("Enter digits 0 to 8 exactly once.")

else:
    print("\nIDS Solution:")

    solution = ids(start)

    display(solution)
```

```
================== RESTART: C:/Users/Student/Desktop/Om/IDS.py ==================
Enter initial state (0 for blank): 123405678

IDS Solution:
Number of moves: 14

1 2 3
4 _ 5
6 7 8

1 2 3
4 5 _
6 7 8

1 2 3
4 5 8
6 7 _

1 2 3
4 5 8
6 _ 7

1 2 3
4 5 8
_ 6 7

1 2 3
_ 5 8
4 6 7

1 2 3
5 _ 8
4 6 7
```